

Capstone Assessment- Spring Boot/Angular Mini ECommerce Application

CONTENTS

1	Problem Statement.....	2
2	Screen Flows.....	3
3	Technical Details.....	11

1 PROBLEM STATEMENT

Develop a mini Ecommerce application using ANGULAR and SpringBoot and MySQL.

The system will consist of three independent microservices:

1. **User Management Microservice:** handles user-related operations
2. **Product Management Microservice:** manages product catalogues and inventory
3. **Order Management Microservice:** processes and manages orders

Primary Actors:

1. Admin: responsible for managing the product catalogue and inventory
2. Customer: browses and purchases products from the catalogue

Basic Flow:

Product Management:

1. Admin adds a new product:
 - Admin submits a new product request with details (name, description, price, quantity)
 - Product service validates the request and creates a new product entity
 - Product service returns a success response to the admin
2. Admin updates an existing product:
 - Admin submits an update product request with updated details (name, description, price, quantity)
 - Product service validates the request and updates the product entity
 - Product service returns a success response to the admin
3. Admin deletes a product:
 - Admin submits a delete product request
 - Product service validates the request and deletes the product entity
 - Product service returns a success response to the admin

User Management:

1. User registration: **admin user creation will be done from back end**
 - User submits a registration request with details (username, email, password)

- User service validates the request and creates a new user entity
2. User profile update:
 - User submits an update profile request with updated details (name, email, address)
 - User service validates the request and updates the user entity

Order Management:

1. Customer places an order:
 - Customer submits an order request with product details and quantity
 - Order service validates the request and communicates with the product service to check available quantity
 - If the requested quantity is available order is created and the quantity for that product will be reduced through Product service.
 - If the requested quantity is not available a message will be sent back to the user saying quantity is not available.
2. Order cancellation:
 - Customer submits an order cancellation request
 - Order service changes the status of the order to cancelled and updates the product quantity accordingly
 - Order service will return a success message to the customer.

Role based access management needs to be implemented so that

- 1) Only admin user can have access to the Product Management functionality.
- 2) Customer can only have access to the orders they have placed.

Scope of the project will be to :

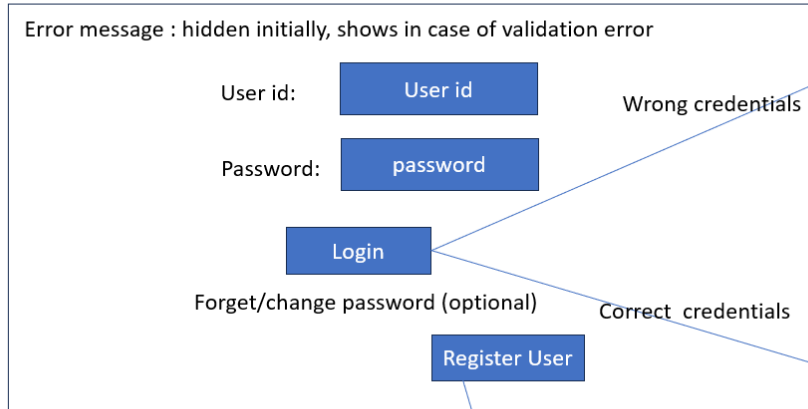
- a) Analyze the requirement.
- b) UI Design.
- c) Database design.
- d) REST endpoint design (i.e designing of REST urls, methods and input data).

2 SCREEN FLOWS

1.

Flow : Admin

sc-01-common –login



Invalid credentials: show message on the top of the login screen saying login unsuccessful : user id or password is wrong. Check for other validations like if you are using email as user id , email format to be checked.

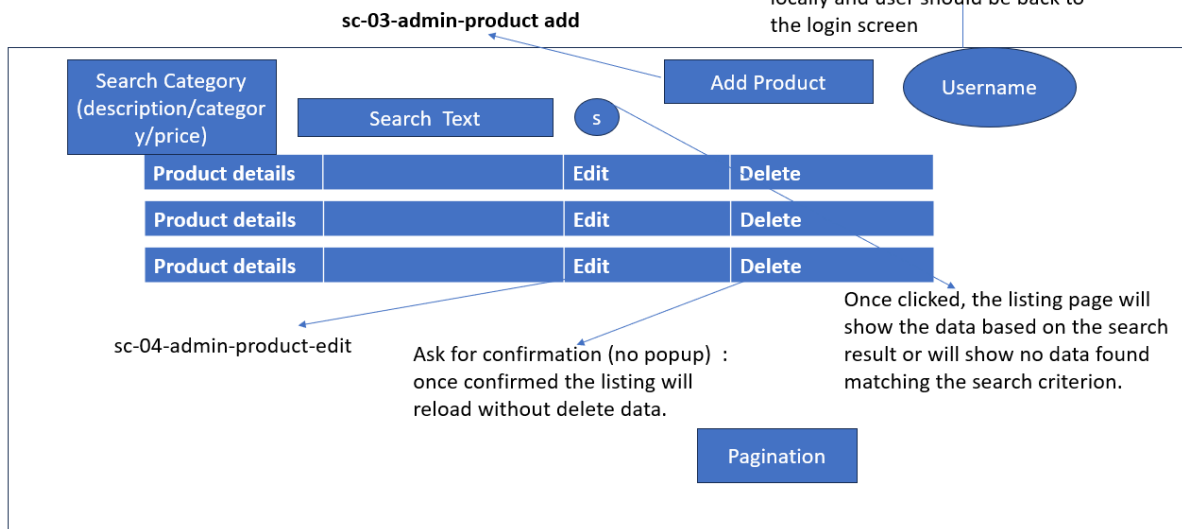
Go to product listing page : **sc-02-admin-product-listing** if the user is admin else go to **sc-05-user-product-catalogue**

sc-09-user-register

2.

Flow : Admin

sc-02-admin-product-listing



There should be logout option on clicking this icon , after logout clear all data like token etc stored locally and user should be back to the login screen

Once clicked, the listing page will show the data based on the search result or will show no data found matching the search criterion.

Ask for confirmation (no popup) : once confirmed the listing will reload without delete data.

3.

Flow : Admin

There should be logout option on clicking this icon , after logout clear all data like token etc stored locally and user should be back to the login screen

sc-03-admin-product-add

Product Name :

Product Category :

Price:

UOM:

QTY:

Username

Check for validations if error show along side the fields (like product name is mandatory, UOM should be number).
If no error go to **sc-02-admin-product-listing**

4.

Flow : Admin

There should be logout option on clicking this icon , after logout clear all data like token etc stored locally and user should be back to the login screen

sc-04-admin-product-edit

Prefill the product details based on the product id.

Product Name :

Product Category :

Price:

UOM:

QTY:

Username

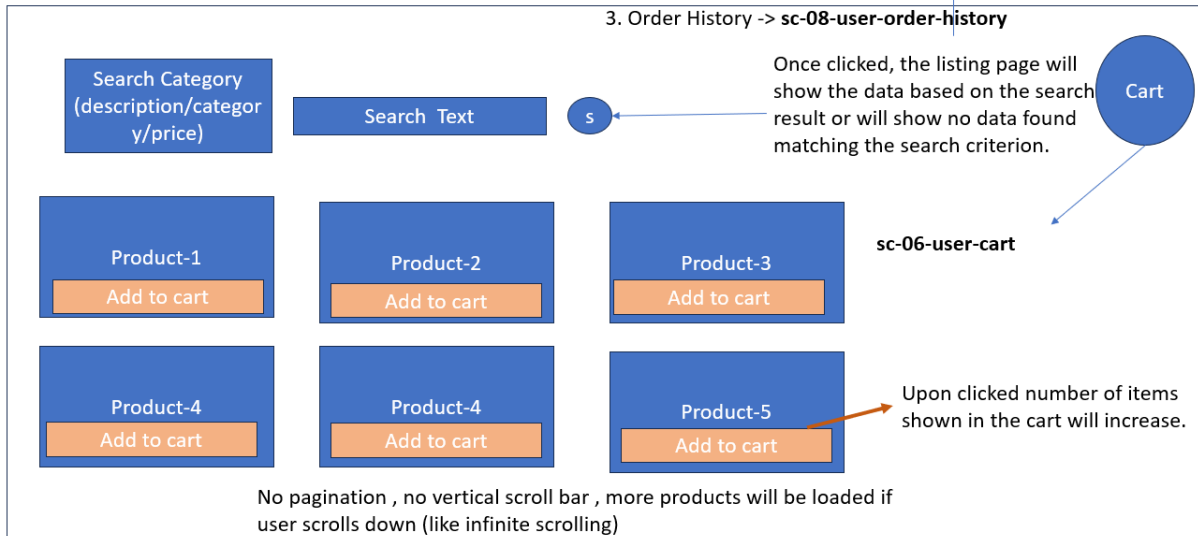
Check for validations if error show along side the fields (like product name is mandatory, UOM should be number).
If no error go to **sc-02-admin-product-listing**

5.

Flow : User
sc-05-user-product-catalogue

- 1: Logout ->sc-01-common –login clearing all data in the local storage
2. Update user profile ->sc-07-user-profile-update

3. Order History -> sc-08-user-order-history



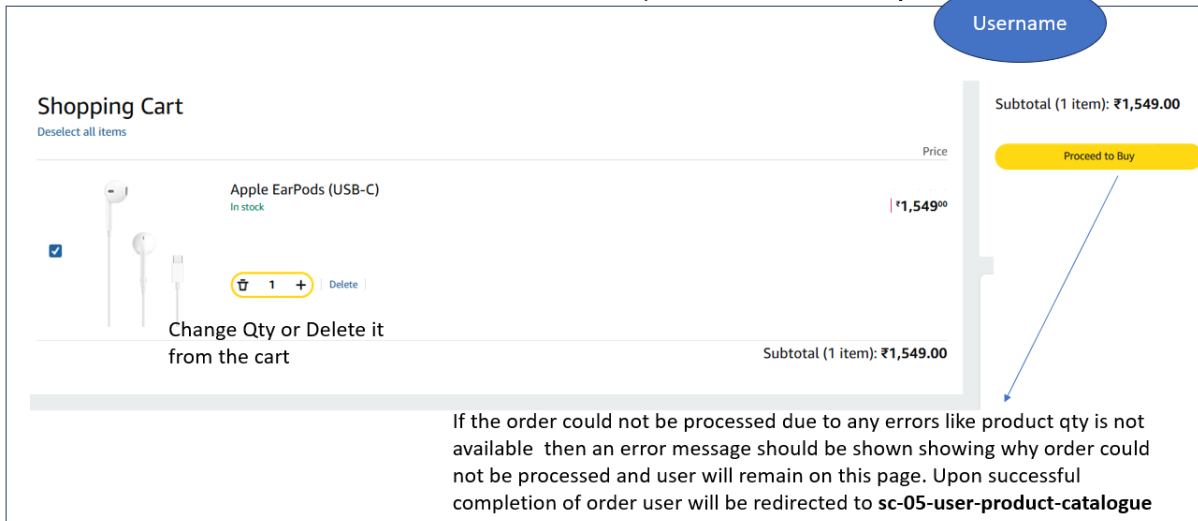
6.

Flow : User

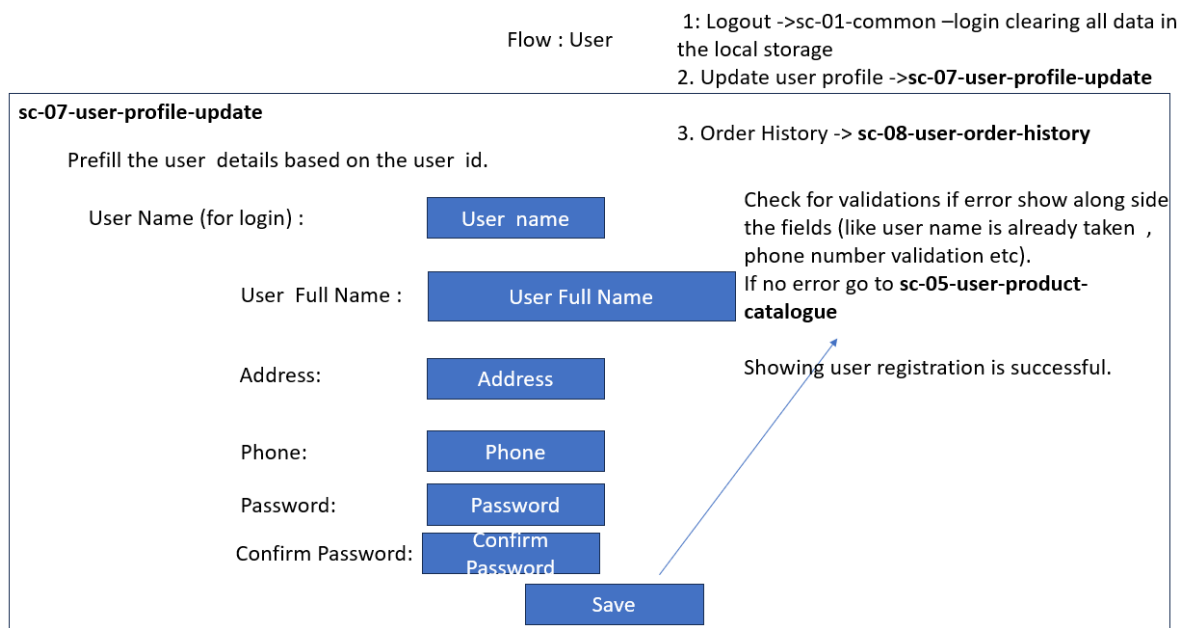
- 1: Logout ->sc-01-common –login clearing all data in the local storage
2. Update user profile ->sc-07-user-profile-update

3. Order History -> sc-08-user-order-history

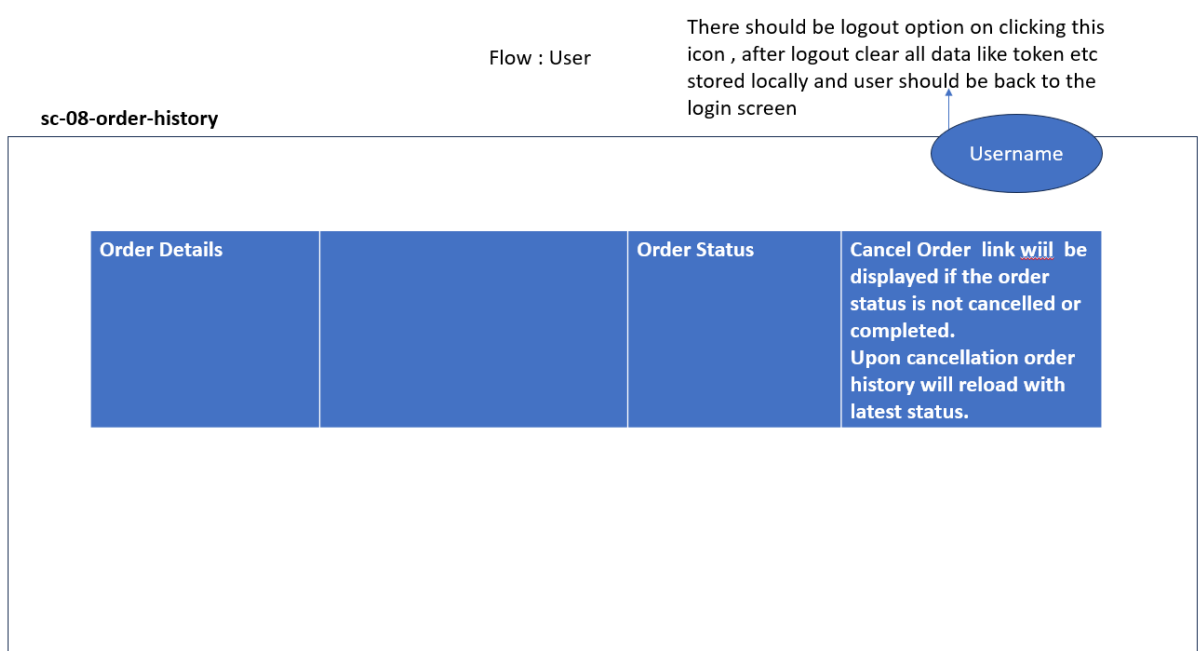
sc-06-user-cart



7.



8.



9.

Flow : Admin

sc-09-user-register

User Name (for login) :	User name	Check for validations if error show along side the fields (like user name is already taken , phone number validation etc). If no error go to sc-01-common –login Showing user registration is successful.
User Full Name :	User Full Name	
Address:	Address	
Phone:	Phone	
Password:	Password	
Confirm Password:	Confirm Password	
Save		

3 API END POINTS

User Management MS:

1. Creation /user (POST)
2. Change user data : /user (PUT)
3. User deletion :/user (DELETE)
4. User List : /user (GET)
5. User detail :/user/{id} (GET)
6. Menu : /user/menu/{id}
7. /user/login (POST)
8. /user/logout (POST)

Product Management MS:

1. Creation /product (POST)
2. Change product data : / product (PUT)
3. Product deletion :/product (DELETE)
4. Product List : /product (GET)
5. Product detail :/ product /{id} (GET)

Order Management (Customer) MS:

There will be two modules in the order management microservice.

Cart :

1. Add Product to Cart : /cart/addProd (User ID/Product and Qty) ->Post

2. Delete products from the Cart (cart item id) ->/cart/deleteProd/{itemid} (Delete)
3. Change the Qty -> /cart/update (Product and Qty) -> PUT
4. View cart ->/cart/{userId} (Get)

Order:

1. Create order /order (Order Details -> get calling cart details) – POST
2. Cancel order /order/{ordelId} –PUT update status field of the order
3. No delete order feature
4. View order /order ->Get list of all orders for all users
5. List /order/{userId} -> List of orders for a particular user ->GET
6. Order details /order/{orderId} ->GET

Note : Add additional API end points if required to cater to all requirements.

4 POJO/ENTITY CLASSES

Order MS:

```
public class User {
    int id;
    String firstName;
    String lastName;
    String emailId;
    String userId;
    String passWord; //encrypted
    String address;
    int userType; //0-admin , 1- customer
    boolean isLoggedIn; //make it false during logout
}
```

Product MS:

```
public class Product {  
    int id;  
    String prodName;  
    String prodDesc;  
    String prodCat;//electronics, apparel, kitchen, books, etc  
    String make;  
    Int availableQty;  
    Double price;  
    String uom;//litre, kg,pc  
    double prodRating;  
    String imageURL;  
    LocalDate dateOfManufacture;  
}
```

Order MS:

```
public class Cart {  
    int id;  
    int userId;  
    Map<Integer,Integer> prodDetails;  
    int totalQty;  
    double totalPrice;  
}  
  
class Order {  
    private int orderId;  
    private int userId;  
    private Map<Integer, Integer> productQuantityMap;  
    private double total Amount;  
    private String OrderStatus;  
    private LocalDateTime orderDate;  
}
```

Note: Add additional classes if required to cater to all requirements.

5 TECHNICAL ARCHITECTURE

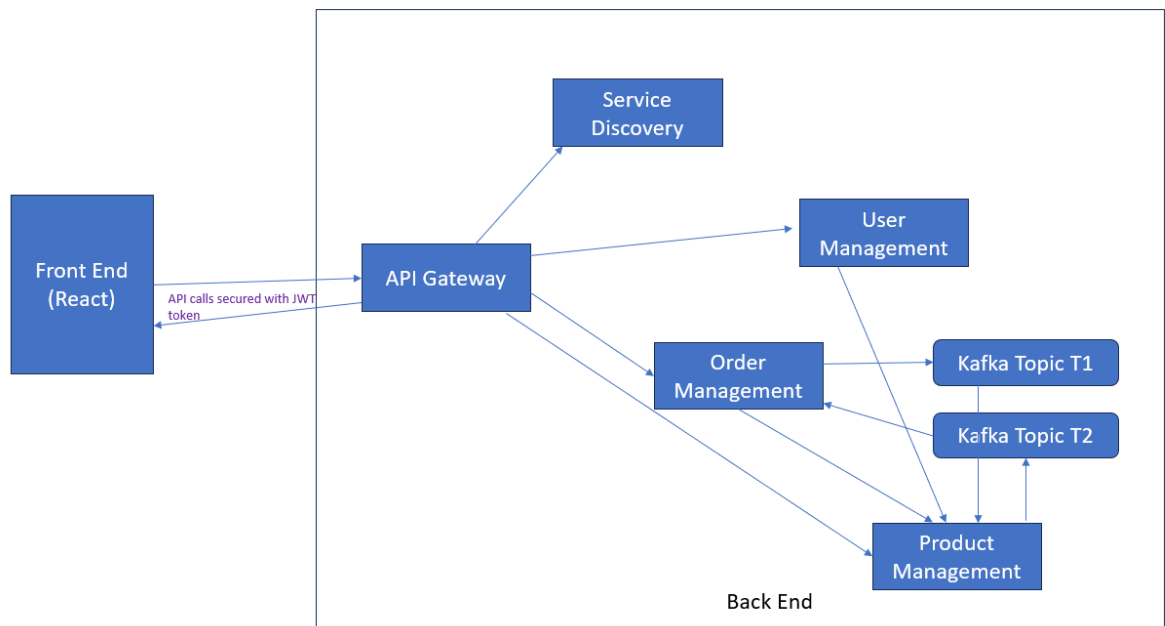


Figure 1 Application Architecture

6 DEVELOPMENT METHODOLOGY

Sprint based development methodology will be followed as below :

Sprint 1: Design Thinking and Backend Foundation :

- Develop wireframes and user stories for the project.
- Set up a Spring Boot application.
- Create RESTful APIs for user management.
- Establish database schema and integrate it with the backend.

Sprint 2: Backend Module:

- Complete User Management API along with Role-Based Access
- Product Catalog API
- Order Management API
- Implement Kafka integration

Sprint 3: Front end Module:

- Build dashboard and all forms for User Management Product Management and Order Management and login.

Swagger Documentation Guideline:

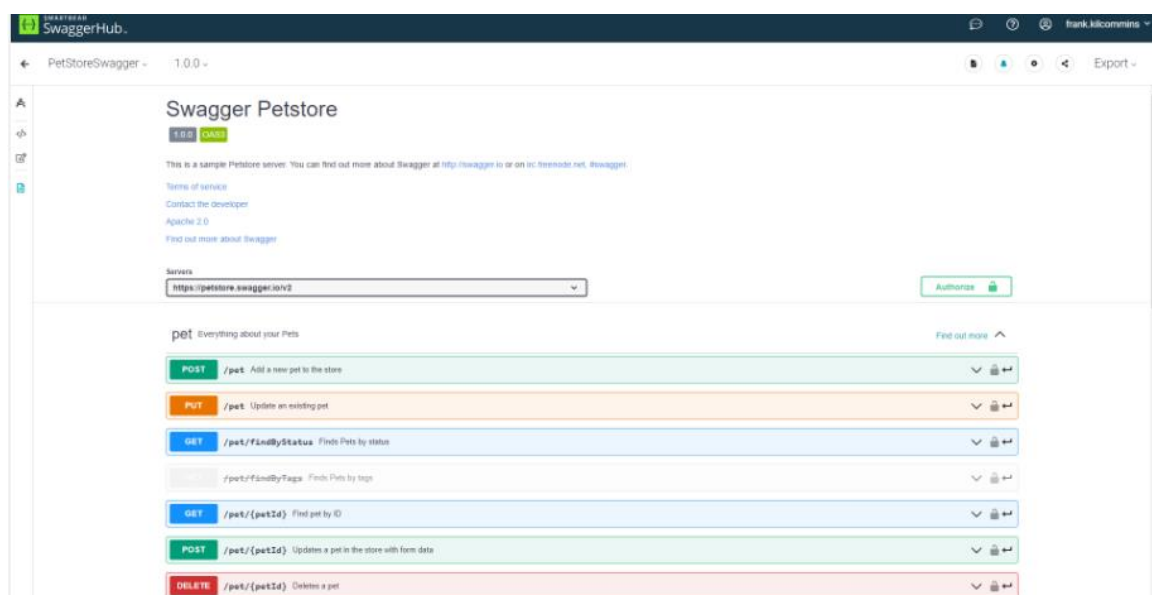
- Use @Tag , @Operation, ApiResponse and @ApiResponse annotations for swagger documentation. A sample class is provided below as reference:

```
@RestController
@RequestMapping("/api/users")
@Tag(name = "User Management", description = "APIs for managing users")
public class UserController {

    @Operation(summary = "Create a new user")
    @ApiResponse(value = {
        @ApiResponse(responseCode = "201", description = "User created successfully"),
        @ApiResponse(responseCode = "400", description = "Invalid user data supplied")
    })
    @PostMapping
    public User createUser(@RequestBody User user) {
        // ... implementation
        return user;
    }

    @Operation(summary = "Get a user by ID")
    @ApiResponse(value = {
        @ApiResponse(responseCode = "200", description = "User found"),
        @ApiResponse(responseCode = "404", description = "User not found")
    })
    @GetMapping("/{id}")
    public User getUserById(@PathVariable Long id) {
        // ... implementation
        return new User(); // dummy return
    }
}
```

The api documentation should be visible through swagger-ui as below (image is for reference only):



UI/UX Guideline:

Core UX Guidelines for UI Development

- Consistency: Maintain consistent design elements (colors, fonts, buttons).
- Clarity: Make UI elements clear and understandable.
- User control: Give users control over interactions.
- Feedback: Provide feedback for user actions (e.g., button clicks).
- Clear messaging: Use clear, concise language for labels, buttons, errors.

UI Design Best Practices

- Simple and intuitive: Minimize complexity in UI.
- Use familiar patterns: Use common UI patterns users are familiar with.
- Responsive design: Ensure UI works well across devices/screen size

General Guidelines:

- Use Java version 17/21.
- Use layering in Microservices.
- Exception handling to be done.
- Use proper package structure like com.wipro.ecom
- Use Swagger for API documentation (optional).
- Angular Routing should be used.
- Usage of Bootstrap is advisable.
- Proper testing to be done.
- Code should be maintained in a Git repository.

Core Technology:

- HTML/CSS/Angular/Spring Boot

Deliverables:

- Screen shots/Videos showing all operations and data in the database.
- The html , css and .js files as source code in proper folder structure.